

SRM UNIVERSITY, ANDHRA PRADESH
SUMMER INTERNSHIP COURSE, June 2026
WEEKLY DIARY REPORT
(B.Tech. STUDENT BATCH 2024-28)

(To be submitted to Faculty Mentor over mail with CC to Industry mentor)

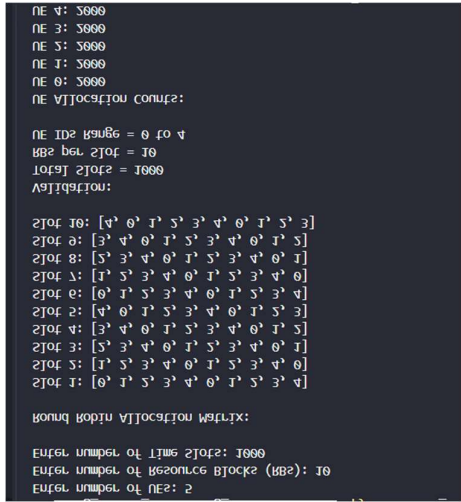
Name of Student: Valluru Komal Sivaram, Earla Bhavyanshi, Makineni Ruthvik Krishna

ID NO: AP24110010764, AP24110010790, AP24110011153

Company Name (For industry internship):

Name of Faculty Mentor: Anil Carie

Week 1 From 25/05/2026 to 29/05/2026		
S. No	Field	Answer
1	Project Title	Resource Scheduling in 5G using Reinforcement Learning
2	Project Description	The project focuses on understanding and implementing resource scheduling techniques in 5G networks. During Week 1, the work included learning Python fundamentals, studying 5G NR concepts, implementing a Round Robin scheduler, and reviewing AI-based scheduling approaches for future research.
3	Outline of the Solution	• Study Python programming fundamentals • Learn core 5G scheduling concepts • Implement Round Robin scheduling algorithm • Analyze limitations and identify scope for AI-based scheduling
4	Design of the Solution	Designed a Python based simulation where network resources are distributed among multiple users using Round Robin scheduling. The scheduler allocates resource blocks cyclically across time slots to ensure fairness.
5	Hardware and Software Requirements to execute the project	Hardware: Laptop/Desktop, Internet access Software: Python 3.11, VS Code, Jupyter Notebook, Git, GitHub
6	Environment setup windows/linux/Raspberry pi/Arduino	Windows Environment Installed Python 3.11, VS Code, Jupyter Notebook, Git and configured project

7	Concepts Used (Functions, header files, data types, and concepts (Loops, arrays, conditional statements, etc.). Explanation of the concepts). For Hardware projects also explain with respect to the code being developed	• Variables and expressions • Conditional statements • Loops and iteration • Functions and modular programming • Arrays/lists for allocation tracking • Python scheduling logic • ☐ 5G concepts: PRB, SINR, QoS, HARQ, Numerology
8	Testing & Validation (Boundary tests and boundaries of inputs. Possible inputs and corresponding outputs).	Tested scheduler using: • 5 users • 10 resource blocks • 100 time slots Validation: • Equal allocation achieved • No user starvation observed • Fair cyclic scheduling confirmed
9	Testing Material (Screenshots of working outputs. Images in case of hardware project)	 <pre> NE 4: 5000 NE 3: 5000 NE 5: 5000 NE 1: 5000 NE 0: 5000 NE allocation config: NE ID2 range = 0 to 4 RBs bgn. slot = 10 total slots = 1000 Allocation: slot 10: [4 0 1 3 2] slot 0: [3 4 0 1 5] slot 8: [5 3 4 0 1] slot 1: [1 5 3 4 0] slot 0: [0 1 5 3 4] slot 2: [4 0 1 5 3] slot 4: [3 4 0 1 5] slot 3: [5 3 4 0 1] slot 5: [1 5 3 4 0] slot 1: [0 1 5 3 4] Round Robin allocation matrix: Enter number of times slots: 1000 Enter number of resource blocks (RBs): 10 Enter number of users: 2 </pre>

		Round Robin Allocation Matrix <table><tr><th>TTI \ UE</th><th>0</th><th>1</th><th>2</th><th>3</th><th>4</th><th>0</th><th>1</th><th>2</th><th>3</th><th>4</th></tr><tr><th>0</th><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td></tr><tr><th>1</th><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td></tr><tr><th>2</th><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td></tr><tr><th>3</th><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td></tr><tr><th>4</th><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td></tr><tr><th>5</th><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td></tr><tr><th>6</th><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td></tr><tr><th>7</th><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td></tr><tr><th>8</th><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td></tr><tr><th>9</th><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td></tr></table>	TTI \ UE	0	1	2	3	4	0	1	2	3	4	0	0	1	2	3	4	0	1	2	3	4	1	0	1	2	3	4	0	1	2	3	4	2	0	1	2	3	4	0	1	2	3	4	3	0	1	2	3	4	0	1	2	3	4	4	0	1	2	3	4	0	1	2	3	4	5	0	1	2	3	4	0	1	2	3	4	6	0	1	2	3	4	0	1	2	3	4	7	0	1	2	3	4	0	1	2	3	4	8	0	1	2	3	4	0	1	2	3	4	9	0	1	2	3	4	0	1	2	3	4
TTI \ UE	0	1	2	3	4	0	1	2	3	4																																																																																																																	
0	0	1	2	3	4	0	1	2	3	4																																																																																																																	
1	0	1	2	3	4	0	1	2	3	4																																																																																																																	
2	0	1	2	3	4	0	1	2	3	4																																																																																																																	
3	0	1	2	3	4	0	1	2	3	4																																																																																																																	
4	0	1	2	3	4	0	1	2	3	4																																																																																																																	
5	0	1	2	3	4	0	1	2	3	4																																																																																																																	
6	0	1	2	3	4	0	1	2	3	4																																																																																																																	
7	0	1	2	3	4	0	1	2	3	4																																																																																																																	
8	0	1	2	3	4	0	1	2	3	4																																																																																																																	
9	0	1	2	3	4	0	1	2	3	4																																																																																																																	
10	User Manual	• Install Python and dependencies • Open project in VS Code • Run round_robin.py • Observe scheduling outputs																																																																																																																									
11	Technical Documentation	The scheduler simulates resource allocation across users using cyclic scheduling logic. Repository structure was organized into scheduler modules, documentation, and future experiment folders.																																																																																																																									
12	References	Survey Paper on Deep Reinforcement Learning for 5G Resource Scheduling, IEEE Xplore, 2024 ☐ Python for Everybody – University of Michigan ☐ 5G Explained – PowerCert Animated Videos																																																																																																																									
13	Daily Work Breakdown																																																																																																																										
	Day 1	Installed Python, VS Code, Jupyter Notebook, Git and created repository structure.																																																																																																																									
	Day 2	Studied 5G NR fundamentals and prepared 5G glossary.																																																																																																																									
	Day 3	Implemented Round Robin Scheduler using Python.																																																																																																																									
	Day 4	Read research paper on AI-based 5G scheduling and prepared summary.																																																																																																																									
	Day 5	Demonstrated scheduler and reviewed throughput, latency and fairness KPIs.																																																																																																																									
Week 2 From 01/06/2026 to 05/06/2026																																																																																																																											
S. No	Field	Answer																																																																																																																									

1	Project Title	Resource Scheduling in 5G using Reinforcement Learning
2	Project Description	This week focused on building the simulation environment required for Reinforcement Learning-based resource scheduling in 5G. A custom Gymnasium environment was created, a Proportional Fair scheduler was implemented, and baseline performance metrics were collected for future comparison.
3	Outline of the Solution	Studied Reinforcement Learning concepts, designed a custom scheduling environment, implemented the Proportional Fair scheduler, and evaluated baseline performance using fairness metrics.
4	Design of the Solution	Designed a Gymnasium-based simulation environment with states, actions, rewards, and transitions. Integrated Round Robin and Proportional Fair schedulers for performance evaluation.
5	Hardware and Software Requirements to execute the project	Hardware: •Laptop/Desktop • Internet Connection Software: •Python3.11 •VSCode •JupyterNotebook •Gymnasium • Git & GitHub
6	Environment setup windows/linux/Raspberry pi/Arduino	Windows Environment Installed and configured Python libraries, Gymnasium framework, and integrated scheduler files for testing and simulation.
7	Concepts Used (Functions, header files, data types, and concepts (Loops, arrays, conditional statements, etc.). Explanation of the concepts). For Hardware projects also explain with respect to the code being developed	Functions Lists and Arrays Conditional Statements Loops Classes and Objects Markov Decision Process (MDP) States, Actions, Rewards Reinforcement Learning Concepts Proportional Fair Scheduling Jain's Fairness Index
8	Testing & Validation (Boundary tests and boundaries of inputs.	Validated the custom environment using reset() and step() functions. Round Robin Results: Total Allocations – 10,000

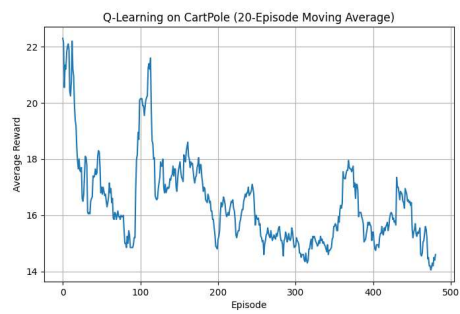
		comparison. The environment will support future Deep Reinforcement Learning experiments.
12	References	□ Sutton, R.S., and Barto, A.G., Reinforcement Learning: An Introduction, MIT Press, 2018. □ David Silver – Reinforcement Learning Course Lectures. □ Reinforcement Learning Specialization – University of Alberta.
13	Daily Work Breakdown	
	Day 1	Completed RL learning and defined environment specifications.
	Day 2	Built custom Gymnasium environment.
	Day 3	Implemented Proportional Fair scheduler.
	Day 4	Collected baseline KPI metrics.
	Day 5	Generated comparison charts and completed mentor demonstration.
Week 3 From 08/06/2026 to 13/06/2026		
S. No	Field	Answer
1	Project Title	Resource Scheduling in 5G using Reinforcement Learning
2	Project Description	This week focused on building the initial Deep Reinforcement Learning pipeline for the project. The work included learning Q-learning and Deep Q-Networks (DQN), integrating Stable-Baselines3 with the scheduling environment, and validating the training process through reward analysis.
3	Outline of the Solution	Studied Deep Reinforcement Learning concepts, implemented Q-learning and DQN methods, integrated Stable-Baselines3 with the custom scheduling environment, and analyzed reward performance.
4	Design of the Solution	Designed a reinforcement learning workflow using Stable-Baselines3 and modified the scheduling environment to support training through observation space and action space compatibility.

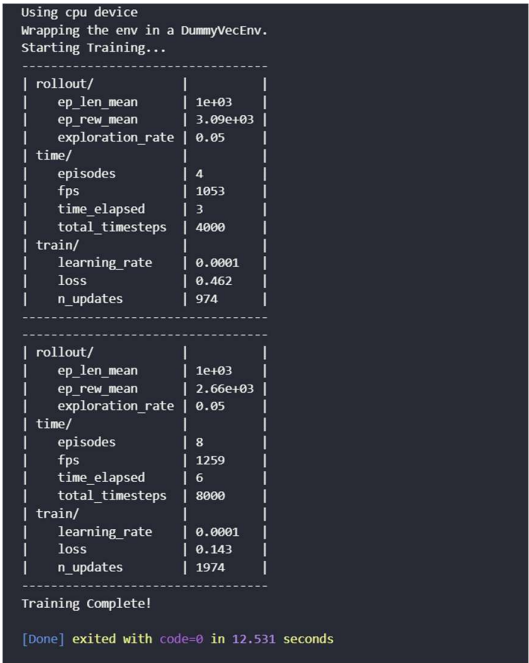
5	Hardware and Software Requirements to execute the project	Hardware: •Laptop/Desktop • Internet Connection Software: •Python3.11 •VSCode •JupyterNotebook •Stable-Baselines3 •Gymnasium • Git & GitHub
6	Environment setup windows/linux/Raspberry pi/Arduino	Windows Environment Configured Stable-Baselines3 and integrated DQN training with the scheduling environment. Verified training execution and environment interaction.
7	Concepts Used (Functions, header files, data types, and concepts (Loops, arrays, conditional statements, etc.). Explanation of the concepts). For Hardware projects also explain with respect to the code being developed	• Functions • Loops • Conditional Statements • Classes and Objects • Q-learning • Deep Reinforcement Learning (DRL) • Deep Q-Network (DQN) • Stable-Baselines3 • Replay Buffer • Target Network • Epsilon-Greedy Policy • Reward Functions • Observation Space and Action Space
8	Testing & Validation (Boundary tests and boundaries of inputs. Possible inputs and corresponding outputs).	Performed testing using: •Tabular Q-learning on CartPole • Stable-Baselines3 DQN training • DQN integration with scheduling environment • Reward monitoring and validation Results: Tabular Q-Learning: Reward improvement observed 500 episodes completed SB3 DQN: Reward improvement observed 10,000 training steps completed Scheduling Environment: Reward curve generated Environment interaction successful No runtime failures observed

9

Testing Material
(Screenshots of working
outputs. Images in case
of hardware project

```
rollout/
| ep_len_mean | 9.79 |
| ep_rew_mean | 9.79 |
| exploration_rate | 0.05 |
time/
| episodes | 960 |
| fps | 944 |
| time elapsed | 10 |
| total_timesteps | 9955 |
train/
| learning_rate | 0.0001 |
| loss | 4.97e-05 |
| n_updates | 2463 |
-----
rollout/
| ep_len_mean | 9.77 |
| ep_rew_mean | 9.77 |
| exploration_rate | 0.05 |
time/
| episodes | 964 |
| fps | 945 |
| time elapsed | 10 |
| total_timesteps | 9991 |
train/
| learning_rate | 0.0001 |
| loss | 0.000123 |
| n_updates | 2472 |
-----
Training Complete!
[Done] exited with code=0 in 28.397 seconds
```



		 <pre> Using cpu device Wrapping the env in a DummyVecEnv. Starting Training... ----- rollout/ ep_len_mean 1e+03 ep_reward_mean 3.09e+03 exploration_rate 0.05 time/ episodes 4 fps 1053 time_elapsed 3 total_timesteps 4000 train/ learning_rate 0.0001 loss 0.462 n_updates 974 ----- ----- ----- rollout/ ep_len_mean 1e+03 ep_reward_mean 2.66e+03 exploration_rate 0.05 time/ episodes 8 fps 1259 time_elapsed 6 total_timesteps 8000 train/ learning_rate 0.0001 loss 0.143 n_updates 1974 ----- ----- Training Complete! [Done] exited with code=0 in 12.531 seconds </pre>
10	User Manual	Step 1: Install required software and libraries. Step 2: Open project in VS Code. Step 3: Execute DQN training file. Step 4: Run scheduling environment. Step 5: Observe reward graph and outputs.
11	Technical Documentation	Implemented the initial DQN training pipeline and integrated it with the custom scheduling environment. The training workflow was validated using reward analysis and environment interaction testing.
12	References	□ V. Mnih et al., “Human-level Control Through Deep Reinforcement Learning,” Nature, 2015. □ Reinforcement Learning Specialization – University of Alberta.
13	Daily Work Breakdown	
	Day 1	Completed RL coursework and implemented Q-learning.
	Day 2	Studied DQN concepts and documented architecture.
	Day 3	Installed Stable-Baselines3 and executed DQN training.
	Day 4	Modified scheduling environment for DQN compatibility.

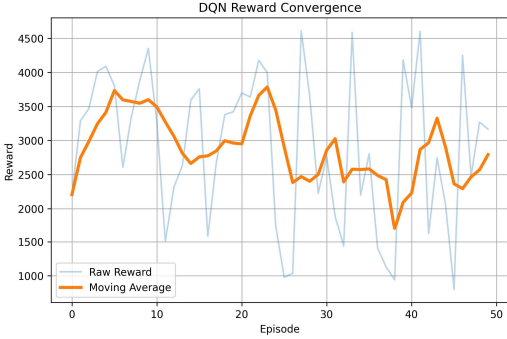
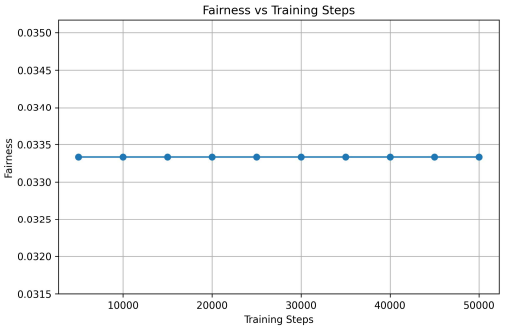
	Day 5	Generated reward curve and completed report documentation.
Week 4 From 15/06/2026 to 19/06/2026		
S. No	Field	Answer
1	Project Title	Resource Scheduling in 5G using Reinforcement Learning
2	Project Description	This week focused on training and evaluating AI-based scheduling approaches for 5G resource allocation. Deep Reinforcement Learning models including DQN and PPO were trained and benchmarked against traditional scheduling methods.
3	Outline of the Solution	Trained DQN and PPO models, evaluated scheduler performance, compared results with Round Robin and Proportional Fair schedulers, and analysed performance using throughput, latency, and Jain's Fairness Index.
4	Design of the Solution	Designed a benchmarking workflow to compare traditional scheduling approaches with Deep Reinforcement Learning models. Implemented evaluation using throughput, latency, and fairness metrics.
5	Hardware and Software Requirements to execute the project	Hardware: • Laptop/Desktop • Internet Connection Software: • Python 3.11 • VS Code • Jupyter Notebook • Stable-Baselines3 • Gymnasium • Git & GitHub
6	Environment setup windows/linux/Raspberry pi/Arduino	Windows Environment Configured DQN and PPO training pipeline and integrated scheduler benchmarking for evaluation.
7	Concepts Used (Functions, header files, data types, and concepts (Loops, arrays, conditional statements, etc.). Explanation of the concepts). For Hardware projects also explain with	• Functions • Conditional Statements • Loops • Classes and Objects • Deep Reinforcement Learning • Deep Q-Network (DQN) • Proximal Policy Optimization (PPO) • Hyperparameter Tuning

	respect to the code being developed	• Reward Functions • Throughput Analysis • Latency Analysis • Jain's Fairness Index																																													
8	Testing & Validation (Boundary tests and boundaries of inputs. Possible inputs and corresponding outputs).	Testing Performed: • Trained DQN with different configurations • Evaluated scheduler performance • Trained PPO model • Compared RR, PF, DQN and PPO approaches Results: Round Robin (RR): Stable throughput and high fairness Proportional Fair (PF): Improved throughput with maintained fairness DQN Scheduler: Adaptive scheduling behaviour with reduced latency PPO Scheduler: Balanced throughput and fairness performance Validation: Benchmark evaluation successfully compared traditional and AI-based scheduling approaches.																																													
9	Testing Material (Screenshots of working outputs. Images in case of hardware project	<table><tr><th>S</th><th>Scheduler</th><th>Mean Throughput</th><th>Throughput Std</th><th>Mean Latency</th><th>Latency Std</th><th>Mean Fairness</th><th>Fairness Std</th><th>Remarks</th></tr><tr><td>1</td><td>Round Robin (RR)</td><td>243.62</td><td>67.21</td><td>468.19</td><td>295.11</td><td>0.7718</td><td>0.1091</td><td>Balanced fairness with moderate throughput</td></tr><tr><td>2</td><td>Proportional Fair (PF)</td><td>110.02</td><td>65.25</td><td>190.24</td><td>140.25</td><td>0.7161</td><td>0.1862</td><td>Lowest latency among baseline schedulers</td></tr><tr><td>3</td><td>Deep Q-Network (DQN)</td><td>17510.00</td><td>7328.70</td><td>57170.00</td><td>19389.20</td><td>0.2000</td><td>0.0000</td><td>Maximized throughput but fairness collapsed</td></tr><tr><td>4</td><td>Proximal Policy Optimization (PPO)</td><td>15680.00</td><td>7780.59</td><td>42180.00</td><td>26465.97</td><td>0.2000</td><td>0.0000</td><td>High throughput with lower latency than DQN but poor fairness</td></tr></table>	S	Scheduler	Mean Throughput	Throughput Std	Mean Latency	Latency Std	Mean Fairness	Fairness Std	Remarks	1	Round Robin (RR)	243.62	67.21	468.19	295.11	0.7718	0.1091	Balanced fairness with moderate throughput	2	Proportional Fair (PF)	110.02	65.25	190.24	140.25	0.7161	0.1862	Lowest latency among baseline schedulers	3	Deep Q-Network (DQN)	17510.00	7328.70	57170.00	19389.20	0.2000	0.0000	Maximized throughput but fairness collapsed	4	Proximal Policy Optimization (PPO)	15680.00	7780.59	42180.00	26465.97	0.2000	0.0000	High throughput with lower latency than DQN but poor fairness
S	Scheduler	Mean Throughput	Throughput Std	Mean Latency	Latency Std	Mean Fairness	Fairness Std	Remarks																																							
1	Round Robin (RR)	243.62	67.21	468.19	295.11	0.7718	0.1091	Balanced fairness with moderate throughput																																							
2	Proportional Fair (PF)	110.02	65.25	190.24	140.25	0.7161	0.1862	Lowest latency among baseline schedulers																																							
3	Deep Q-Network (DQN)	17510.00	7328.70	57170.00	19389.20	0.2000	0.0000	Maximized throughput but fairness collapsed																																							
4	Proximal Policy Optimization (PPO)	15680.00	7780.59	42180.00	26465.97	0.2000	0.0000	High throughput with lower latency than DQN but poor fairness																																							

		<pre> rollout/ ep_len_mean 1e+03 ep_rew_mean 2.67e+03 exploration_rate 0.05 time/ episodes 44 fps 1610 time_elapsed 27 total_timesteps 44000 train/ learning_rate 0.0001 loss 0.107 n_updates 10974 ----- ----- rollout/ ep_len_mean 1e+03 ep_rew_mean 2.79e+03 exploration_rate 0.05 time/ episodes 48 fps 1612 time_elapsed 29 total_timesteps 48000 train/ learning_rate 0.0001 loss 0.114 n_updates 11974 ----- ----- Training complete! [Done] exited with code=0 in 38.684 seconds </pre>
10	User Manual	Step 1: Install required libraries Step 2: Open project in VS Code Step 3: Execute DQN and PPO training scripts Step 4: Run benchmark evaluation Step 5: Observe throughput, latency and fairness results
11	Technical Documentation	Developed and validated a benchmarking framework for comparing Round Robin, Proportional Fair, DQN and PPO scheduling methods. Performance was analysed using selected network KPIs.
12	References	□ Deep Reinforcement Learning for Wireless Resource Scheduling, IEEE, 2024. □ Schulman et al., Proximal Policy Optimization Algorithms, 2017.
13	Daily Work Breakdown	
	Day 1	Trained DQN and performed hyperparameter tuning.
	Day 2	Reviewed DRL scheduling research and compared environment design.
	Day 3	Built evaluation workflow and generated KPI comparison.
	Day 4	Implemented PPO training and benchmark evaluation.

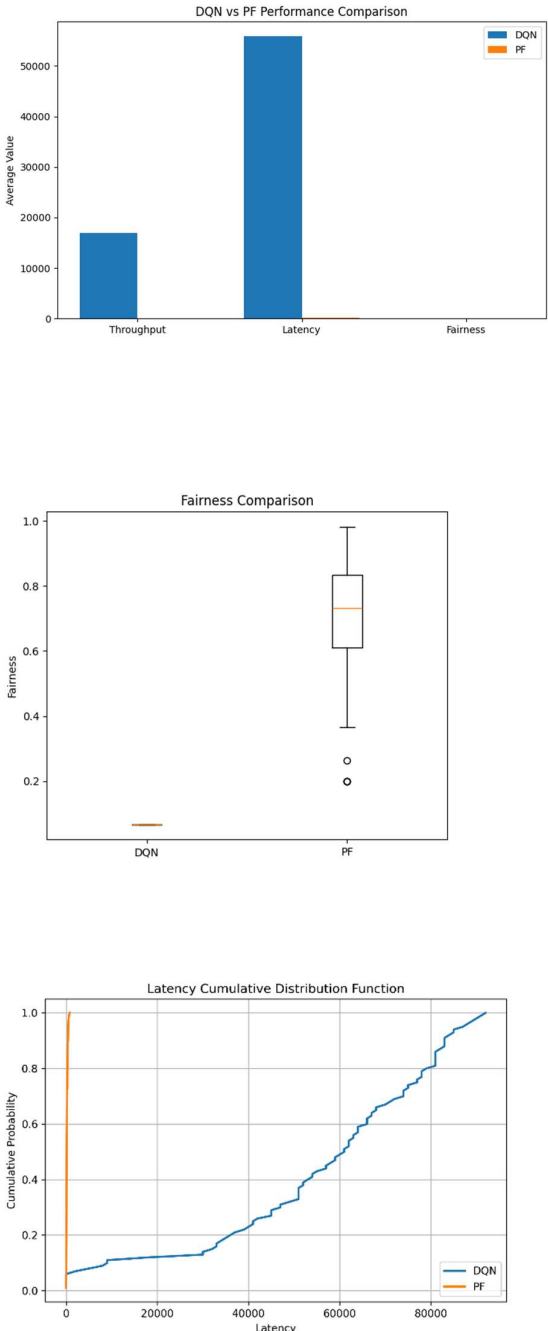
	Day 5	Presented benchmark results and completed report documentation.
Week 5 From 22/06/2026 to 27/06/2026		
S. No	Field	Answer
1	Project Title	Resource Scheduling in 5G using Reinforcement Learning
2	Project Description	This week focused on evaluating the performance and stability of the Deep Reinforcement Learning-based scheduler under different network conditions. Scenario-based evaluation, ablation studies, and convergence analysis were performed to validate scheduling effectiveness and learning behaviour.
3	Outline of the Solution	Evaluated DQN performance under multiple traffic scenarios, analysed QoS-aware reward functions, performed ablation studies, conducted convergence analysis, and validated benchmark results.
4	Design of the Solution	Designed a scenario-based evaluation framework to test scheduler behaviour under varying network loads. Implemented ablation experiments and convergence analysis to study the contribution of DQN components and training stability.
5	Hardware and Software Requirements to execute the project	Hardware: • Laptop/Desktop • Internet Connection Software: • Python 3.11 • VS Code • Jupyter Notebook • Stable-Baselines3 • Gymnasium • Git & GitHub
6	Environment setup windows/linux/Raspberry pi/Arduino	Windows Environment Configured scenario-based testing and integrated convergence analysis with scheduler evaluation workflow.
7	Concepts Used (Functions, header files, data types, and concepts (Loops, arrays, conditional statements, etc.). Explanation of the	• Functions • Classes and Objects • Conditional Statements • Loops • Deep Q-Network (DQN) • QoS-aware Reward Functions

	concepts). For Hardware projects also explain with respect to the code being developed	• Scenario-Based Evaluation • Ablation Study • Convergence Analysis • Replay Buffer • Target Network • Throughput Analysis • Latency Analysis • Jain’s Fairness Index																														
8	Testing & Validation (Boundary tests and boundaries of inputs. Possible inputs and corresponding outputs).	Testing Performed: • Evaluated DQN under low, medium, and high network load scenarios • Analysed QoS-aware reward behaviour • Performed ablation study by removing DQN components • Conducted convergence analysis using reward trends Results: Round Robin: Lower throughput under high traffic load Proportional Fair: Improved throughput with maintained fairness DQN Scheduler: Improved adaptive scheduling and stable performance DQN without Replay Buffer: Reduced throughput and increased latency DQN without Target Network: Less stable learning and lower fairness Validation: Scenario evaluation and convergence analysis confirmed that DQN performance depends strongly on training configuration and scheduler design.																														
9	Testing Material (Screenshots of working outputs. Images in case of hardware project	<table><tr><td>1</td><td>Configuration</td><td>Throughput</td><td>Latency</td><td>Fairness</td><td>Observation</td></tr><tr><td>2</td><td>Full DQN</td><td>16630.00</td><td>67140.00</td><td>0.0667</td><td>Baseline configuration</td></tr><tr><td>3</td><td>No Replay Buffer</td><td>16630.00</td><td>45490.00</td><td>0.0667</td><td>Minimal impact on throughput</td></tr><tr><td>4</td><td>No Target Network</td><td>16600.00</td><td>38470.00</td><td>0.0667</td><td>Similar performance to baseline</td></tr><tr><td>5</td><td>Simplified Reward</td><td>16850.00</td><td>58240.00</td><td>0.0667</td><td>Reward simplification had limited impact</td></tr></table>	1	Configuration	Throughput	Latency	Fairness	Observation	2	Full DQN	16630.00	67140.00	0.0667	Baseline configuration	3	No Replay Buffer	16630.00	45490.00	0.0667	Minimal impact on throughput	4	No Target Network	16600.00	38470.00	0.0667	Similar performance to baseline	5	Simplified Reward	16850.00	58240.00	0.0667	Reward simplification had limited impact
1	Configuration	Throughput	Latency	Fairness	Observation																											
2	Full DQN	16630.00	67140.00	0.0667	Baseline configuration																											
3	No Replay Buffer	16630.00	45490.00	0.0667	Minimal impact on throughput																											
4	No Target Network	16600.00	38470.00	0.0667	Similar performance to baseline																											
5	Simplified Reward	16850.00	58240.00	0.0667	Reward simplification had limited impact																											

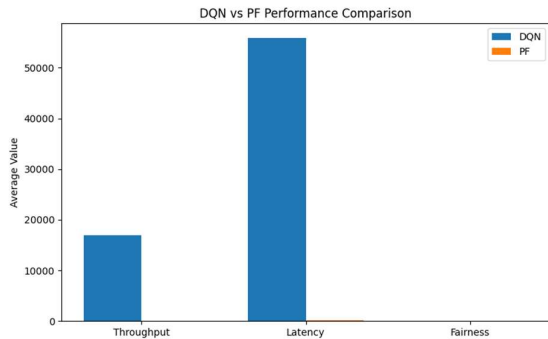
		 
10	User Manual	Step 1: Install required libraries Step 2: Run scenario evaluation files Step 3: Execute DQN scheduler Step 4: Perform ablation testing Step 5: Observe reward and KPI outputs
11	Technical Documentation	Developed and validated a scenario-based evaluation framework for DQN scheduling. Performed ablation and convergence studies to evaluate model reliability and scheduler performance.
12	References	□ Deep Reinforcement Learning for Wireless Resource Scheduling, IEEE, 2024. □ Sutton and Barto, Reinforcement Learning: An Introduction. □ Deep Q-Learning research materials and benchmark evaluation notes.
13	Daily Work Breakdown	
	Day 1	Evaluated DQN under low, medium, and high traffic scenarios.
	Day 2	Analysed QoS-aware reward functions and scheduling behaviour.

	Day 3	Performed ablation study and measured component impact.
	Day 4	Conducted convergence analysis using reward trends.
	Day 5	Validated final results and completed documentation.
Week 6 From 29/06/2026 to 03/07/2026		
S. No	Field	Answer
1	Project Title	Resource Scheduling in 5G using Reinforcement Learning
2	Project Description	This week focused on validating the performance of the AI-based 5G resource scheduling system through statistical analysis, preparing publication-quality figures, and drafting research paper content. Experimental results were verified using statistical methods, and the Related Work section was prepared for the research paper.
3	Outline of the Solution	Re-ran scheduling experiments, performed statistical significance testing, generated publication-quality graphs, reviewed IEEE research papers, and prepared research paper content.
4	Design of the Solution	Designed a validation framework to compare Round Robin, Proportional Fair, PPO, and DQN scheduling algorithms using statistical analysis and performance metrics. Publication-quality figures were created for research paper documentation.
5	Hardware and Software Requirements to execute the project	Hardware: • Laptop/Desktop • Internet Connection Software: • Python 3.11 • VS Code • Jupyter Notebook • Stable-Baselines3 • Gymnasium • Matplotlib • Git & GitHub
6	Environment setup windows/linux/Raspberry pi/Arduino	Windows Environment Configured the evaluation environment to perform statistical analysis, generate publication-quality graphs, and validate experimental results for different scheduling algorithms.

7	Concepts Used (Functions, header files, data types, and concepts (Loops, arrays, conditional statements, etc.). Explanation of the concepts). For Hardware projects also explain with respect to the code being developed	• Functions • Classes and Objects • Conditional Statements • Loops • Deep Reinforcement Learning (DRL) • Deep Q-Network (DQN) • Proximal Policy Optimization (PPO) • Statistical Analysis • Wilcoxon Signed-Rank Test • Mean and Standard Deviation • Throughput Analysis • Latency Analysis • Jain's Fairness Index • Data Visualization using Matplotlib
8	Testing & Validation (Boundary tests and boundaries of inputs. Possible inputs and corresponding outputs).	Testing Performed: • Re-ran experiments using multiple random seeds. • Calculated mean and standard deviation for throughput, latency, and fairness. • Performed Wilcoxon Signed-Rank statistical tests. • Compared Round Robin, Proportional Fair, PPO, and DQN schedulers. • Generated publication-quality graphs and comparison charts. Results: Round Robin (RR): Throughput – 251.73 Latency – 509.25 Jain's Fairness Index – 0.7905 Proportional Fair (PF): Throughput – 112.50 Latency – 215.22 Jain's Fairness Index – 0.7239 DQN (30 UE): Throughput – 17910.00 Latency – 64490.00 Jain's Fairness Index – 0.0333 Validation: The experimental results were successfully validated through statistical analysis. The DQN scheduler achieved the highest throughput among the evaluated methods, and the analysis confirmed the effectiveness of Deep Reinforcement Learning for adaptive resource scheduling.

9	Testing Material (Screenshots of working outputs. Images in case of hardware project)	 The figure consists of three subplots comparing DQN and PF algorithms: DQN vs PF Performance Comparison: A bar chart showing Average Value for Throughput, Latency, and Fairness. DQN (blue) has a high Latency (~55000) and a moderate Throughput (~18000). PF (orange) has very low values for all three metrics. Fairness Comparison: A box plot showing Fairness for DQN and PF. DQN has a very low median fairness (near 0.05), while PF has a higher median fairness (around 0.75) with significant outliers. Latency Cumulative Distribution Function: A CDF plot showing Cumulative Probability vs Latency. PF (orange) shows a very sharp rise at low latency, indicating low latency for most cases. DQN (blue) shows a much more gradual increase, indicating higher and more variable latency.
10	User Manual	Step 1: Install the required Python libraries. Step 2: Execute the scheduler evaluation scripts. Step 3: Run the statistical analysis program. Step 4: Generate graphs using Matplotlib. Step 5: Review the performance metrics and comparison results.
11	Technical Documentation	Validated the performance of AI-based scheduling algorithms using statistical analysis and significance testing.

		Generated publication-quality visualizations and prepared documentation for the conference paper.
12	References	□ V. Mnih et al., "Human-level Control through Deep Reinforcement Learning," <i>Nature</i>, 2015. □ J. Schulman et al., "Proximal Policy Optimization Algorithms," arXiv:1707.06347, 2017. □ IEEE papers on AI-based 5G Resource Scheduling.
13	Daily Work Breakdown	
	Day 1	Re-ran scheduling experiments and calculated mean and standard deviation for performance metrics.
	Day 2	Performed Wilcoxon Signed-Rank statistical analysis and updated comparison tables.
	Day 3	Reviewed IEEE research papers and drafted the Related Work section.
	Day 4	Created publication-quality graphs, convergence plots, and KPI comparison figures.
	Day 5	Finalized figure captions, organized project documentation, and prepared materials for mentor review.
Week 7 From 06/07/2026 to 10/07/2026		
S. No	Field	Answer
1	Project Title	Resource Scheduling in 5G using Reinforcement Learning
2	Project Description	During Week 7, our team focused on preparing the first draft of the research paper by documenting the proposed DRL-based 5G scheduling methodology, organizing experimental results, and drafting the introduction, methodology, results, and conclusion sections.
3	Outline of the Solution	Prepared a structured IEEE-style research paper describing the proposed DRL-based 5G scheduler, including the methodology, experimental results, performance analysis, and research contributions.
4	Design of the Solution	Designed the research paper following the IEEE format, including the system architecture, methodology, experimental results, performance comparison, and conclusion.

5	Hardware and Software Requirements to execute the project	Hardware: Laptop/PC with internet connection. Software: Python, VS Code, Overleaf (LaTeX), Stable-Baselines3, Gymnasium, Matplotlib, GitHub.												
6	Environment setup windows/linux/Raspberry pi/Arduino	Windows Environment Windows operating system with Python, VS Code, Overleaf, GitHub, Stable-Baselines3, Gymnasium, and required Python libraries installed.												
7	Concepts Used (Functions, header files, data types, and concepts (Loops, arrays, conditional statements, etc.). Explanation of the concepts). For Hardware projects also explain with respect to the code being developed	Used concepts such as Deep Reinforcement Learning (DQN/PPO), 5G resource scheduling, QoS-aware scheduling, state-action-reward representation, performance evaluation, IEEE paper formatting, Overleaf (LaTeX), and data visualization using Matplotlib												
8	Testing & Validation (Boundary tests and boundaries of inputs. Possible inputs and corresponding outputs).	Verified the completeness and consistency of the research paper by reviewing the methodology, figures, tables, and experimental results. Cross-checked that all performance metrics and discussions accurately reflected the outcomes obtained in previous weeks.												
9	Testing Material (Screenshots of working outputs. Images in case of hardware project)	 <table border="1"> <caption>DQN vs PF Performance Comparison</caption> <thead> <tr> <th>Metric</th> <th>DQN (Average Value)</th> <th>PF (Average Value)</th> </tr> </thead> <tbody> <tr> <td>Throughput</td> <td>~17000</td> <td>~1000</td> </tr> <tr> <td>Latency</td> <td>~55000</td> <td>~1000</td> </tr> <tr> <td>Fairness</td> <td>~1000</td> <td>~1000</td> </tr> </tbody> </table>	Metric	DQN (Average Value)	PF (Average Value)	Throughput	~17000	~1000	Latency	~55000	~1000	Fairness	~1000	~1000
Metric	DQN (Average Value)	PF (Average Value)												
Throughput	~17000	~1000												
Latency	~55000	~1000												
Fairness	~1000	~1000												
10	User Manual	Open the research paper in Overleaf or a PDF viewer to review the methodology, system design, experimental results, figures, and conclusions. Navigate through each section for a complete understanding of the proposed DRL-based 5G scheduling approach.												
11	Technical Documentation	Prepared technical documentation describing the DRL-based scheduling methodology, system architecture, experimental setup, performance analysis, and IEEE paper structure.												
12	References	□ IEEE Xplore Digital Library (Research papers on 5G Resource Scheduling)												

		☐ Stable-Baselines3 Documentation ☐ Gymnasium Documentation ☐ Overleaf IEEE Conference Template ☐ Python and Matplotlib Documentation
13	Daily Work Breakdown	
	Day 1	Set up the IEEE paper template, prepared the paper outline, and designed the system architecture diagram.
	Day 2	Drafted the methodology section, including the system model, reward function, and DRL scheduler workflow.
	Day 3	Organized experimental results, inserted graphs and tables, and prepared the Results and Discussion section
	Day 4	Wrote the Introduction, Abstract, and Conclusion, ensuring proper flow and formatting.
	Day 5	Reviewed the complete paper, verified references and formatting, and prepared the first draft for mentor review.